

Ödev 10: Doğruyu Söyleyen Bir Config Yükleyici

Birkaç farklı biçimde başarısız olan bir şey kurun, sonra üç yanlış kullanımı yeniden üretin ki her birini bir kez kazara değil bilerek yazmış olun.

Birinci Bölüm: Yükleyici

`ConfigLoader.java` dosyasını klasik biçimde oluşturun.

`key=value` satırlarından ayrıştırılmış bir `Map<String, String>` döndüren `load(String name)` metodu yazın. Üç dosyayı taklit edin:

Ad	Davranış
<code>app.conf</code>	<code>host=localhost</code> ve <code>port=8080</code> döndürür
<code>missing.conf</code>	okuyucu <code>IOException</code> fırlatır
<code>broken.conf</code>	ayrıştırılır, ama <code>port</code> bir sayı değil

Şu özelliklere sahip bir `ConfigException` yazın:

- denetlenen** olsun, burada denetlenenin neden doğru seçim olduğunu söyleyen bir yorumla
- kaynak dosya adını** mesaja gömülü değil bir alan olarak taşısın
- `Throwable` sebep alan bir kurucusu olsun

`main` içinde üç dosyayı da yükleyin. Her başarısızlık için mesajı, kaynağı ve sebebi yazdırın.

`broken.conf` durumu ilginç olanı. `Integer.parseInt` *denetlenmeyen* bir `NumberFormatException` fırlatır. Onun olduğu gibi kaçmasına mı izin vereceğinize yoksa dönüştüreceklerinize mi karar verin ve seçiminizi bir yorumda gerekçelendirin.

İkinci Bölüm: Üç Yanlış Kullanım

Bunların her birini iki kez yazın, bir kez bozuk bir kez düzeltilmiş, ve her iki sonucu da yazdırın.

1. Boş catch bloğu. Bir şeyi yakalayın ve hiçbir şey yapmayın. Sonra düzeltin.

Bir yorumda, bunun programın çökmesine izin vermekten neden daha kötü olduğunu kendi kelimelerinizle açıklayın.

2. `finally` içinde `return`. `try` içinden fırlatın, `finally` içinden dönün, ve istisnanın yok olduğunu gösterin. Sonra düzeltin.

Bunun boş catch bloğunun kılık değiştirmiş hâli olduğunu açıklayın.

3. Çok geniş Exception yakalamak. Bir ayrıştırma hatasını ele almayı *amaçlayan* ama asıl hatası bir `NullPointerException` olan bir metot yazın ve işleyicinin onu yanlış raporladığını gösterin. Sonra düzeltin.

Geniş yakalamanın, hata ayıklamak zorunda kalan kişiye neye mal olduğunu açıklayın.

Üçüncü Bölüm: Kaynaklar

Tek bir try-with-resources içinde iki `AutoCloseable` kaynak açın. **Her iki** `close()` metodunu fırlatacak şekilde yazın, ve **gövdeyi** de fırlatacak şekilde.

Sonucu yakalayın ve yakaladığınız istisnayı artı `getSuppressed()` içeriğini yazdırın.

Sonra şunları kapsayan bir yorum yazın:

1. Hangi kaynağın önce kapandığı ve bu sıranın neden doğru olduğu
2. Hangi istisnayı yakaladığınız ve diğer ikisine ne olduğu
3. Bir `finally` bloğundaki elle yazılmış temizliğin bunun yerine ne yapacağı

Kabul Kriterleri

- ☐ `java ConfigLoader.java` ile çalışıyor
- ☐ `ConfigException` denetlenen, kaynağı alan olarak taşıyor ve sebep kurucusu var
- ☐ Üç config durumu da çalıştırılıyor ve sebebini yazdırıyor
- ☐ Bir yorum `ConfigException` için denetleneni gerekçelendiriyor
- ☐ Bir yorum `NumberFormatException` kararını gerekçelendiriyor
- ☐ Üç yanlış kullanım da bozuk ve düzeltilmiş hâlde görünüyor, çıktı farkı gösteriyor
- ☐ Her yanlış kullanımın kendi kelimelerinizle bir açıklaması var
- ☐ try-with-resources çıktısı ters kapatma sırasını ve iki bastırılan istisnayı gösteriyor
- ☐ Son sürümde, bilerek yanlış kullanım olarak etiketlenen hariç hiçbir boş catch bloğu kalmamış

İleri Seviye

`load` metodunu hiç fırlatmayacak, bunun yerine Modül 09'un ileri seviye bölümündeki gibi mühürlü bir `Result` tipi döndürecek şekilde yeniden yazın.

Sonra kısa bir karşılaştırma yazın. Çağıranın hangisini yanlış yapması daha kolay? Hangisi çağrı yerinde daha iyi okunuyor? Hangisi bir stream içindeki lambda'dan çağrılmaya dayanıyor, ve Modül 10'un denetlenen istisnalar ile stream API'si hakkında söyledikleri göz önüne alındığında bu neden önemli?

İpucu, bastırılan liste boş geliyorsa

Bastırılan istisnalar yalnızca **gövde** de fırlattığında ortaya çıkar. Gövde normal biterse ve bir `close()` fırlatırsa, o kapatma istisnası asıl istisna olur ve onu bastıracak bir şey kalmaz.